Universidad de La Laguna

Pruebas Automatizadas E2E con Playwright

Sistemas y Tecnologías Web: Cliente

Borja Medina Ramos

Diseño de casos de prueba

En primer lugar, hemos diseñado una batería de pruebas implementadas donde estudiaremos diferentes casos de uso de la aplicación TodoMVC evaluando y comprobando el correcto funcionamiento del aplicativo web:

	Contenido
Identificador	**CP01**
Descripción	**Verificación del título y URL de página**
Precondiciones	**Tener el enlace de la página y estar disponible.**
Pasos	**Ir a la página web y posteriormente se verifica si contiene el título y la una URL.**
Resultado Esperado	**El resultado debe ser verdadero porque contiene una URL y su título como *React • TodoMVC***
Aserciones	**toHaveTitle() y toHaveURL()**

	Contenido
Identificador	**CP02**
Descripción	**Nueva tarea y coincide con el texto indicado.**
Precondiciones	**Para la adición de un nuevo elemento, la aplicación está abierta. Para la comprobación del contenido, debe haberse añadido la tarea previamente.**
Pasos	**Ir a la página web, rellenamos la casilla de texto y presionamos la tecla ENTER. Una vez insertado el elemento, comprobamos si el texto coincide con el introducido previamente.**
Resultado Esperado	**El resultado debe crear una lista de un sólo elemento cuyo texto debe coincidir con el introducido al crear la tarea.**
Aserciones	**toBe(), toMatch()**

	Contenido
Identificador	**CP03**
Descripción	**Eliminación de una tarea**
Precondiciones	**La lista debe contener al menos un elemento para poder ser eliminado.**
Pasos	**Se accede a la página web, se introduce una nuevo elemento y se elimina.**
Resultado Esperado	**Ese ítem o elemento se ha desacoplado del DOM de la página.**
Aserciones	**toHaveText(), toBeHidden(), toBeVisible() y toHaveCount(0) para comprobar que se desacopla del DOM.**

	Contenido
Identificador	**CP04**
Descripción	**Verificación de contenido en algún elemento de la lista**
Precondiciones	**La página de TodoMVC debe cargarse correctamente y el input debe estar visible y habilitado. Asimismo se deben insertar varios elementos para comprobar si alguno de estos contiene un texto en concreto.**
Pasos	**Se accede a la página web, se introducen cuatro tareas mediante la caja de introducción de texto, se comprueba que la lista contiene exactamente tantos elementos como tareas introducidas y se verifica que al menos uno de ellos contiene la palabra "Docker" sin distinguir mayúsculas.**
Resultado Esperado	**El número de elementos de la lista coincide con el tamaño del array y se encuentra el texto introducido**
Aserciones	**toHaveCount() sobre el elemento lista para validar el total de tareas, y not.toHaveCount(0) para confirmar que existe al menos una coincidencia**

	Contenido
Identificador	CP05
Descripción	Cambio de URL
Precondiciones	La página de TodoMVC debe cargarse correctamente
Pasos	Se accede a la página web y se obtiene la URL actual.
Resultado Esperado	La URL de acceso a la página no coincide con la URL actual.
Aserciones	toBeFalsy()

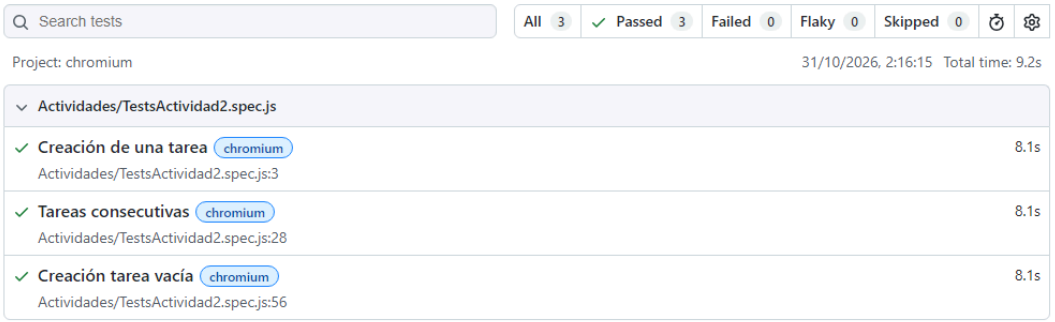
	Contenido
Identificador	CP06
Descripción	No permitir tareas vacías
Precondiciones	Aplicación abierta, lista vacía y campo de entrada disponible.
Pasos	Se accede a la página web, se introducen únicamente espacios en el campo de texto y se pulsa Enter para intentar crear una tarea. Posteriormente, se comprueba que no se ha añadido ningún elemento a la lista y que el contador de tareas pendientes no aparece.
Resultado Esperado	La prueba debe finalizar correctamente, ya que la aplicación no debe permitir crear tareas cuyo contenido esté formado únicamente por espacios. La lista debe contener 0 elementos y el contador tiene uqe estar oculto.
Aserciones	toHaveCount(0) para comprobar que no se ha añadido ninguna tarea y toBeHidden() para verificar que el contador no aparece.

	Contenido
Identificador	**CP07**
Descripción	**Verificar tarea desmarcada**
Precondiciones	**Aplicación abierta, lista vacía y campo de entrada disponible. Debe haber una tarea completada y desmarcarla en el apartado de Completed.**
Pasos	**Se accede a la página web, se crea la tarea una tarea, se marca como completada, se accede a la pestaña Completed, se comprueba que la lista contiene 1 elemento, se desmarca el checkbox y se vuelve a comprobar que la lista contiene 1 elemento.**
Resultado Esperado	**La prueba debe fallar en la última aserción: tras desmarcar la tarea, esta deja de estar completada y desaparece de Completed, por lo que la lista contiene 0 elementos en lugar de 1.**
Aserciones	**toHaveCount(1) de nuevo tras click() (fallo intencionado: esperado 1, recibido 0)**

Actividad 2: Creación y validación de tareas

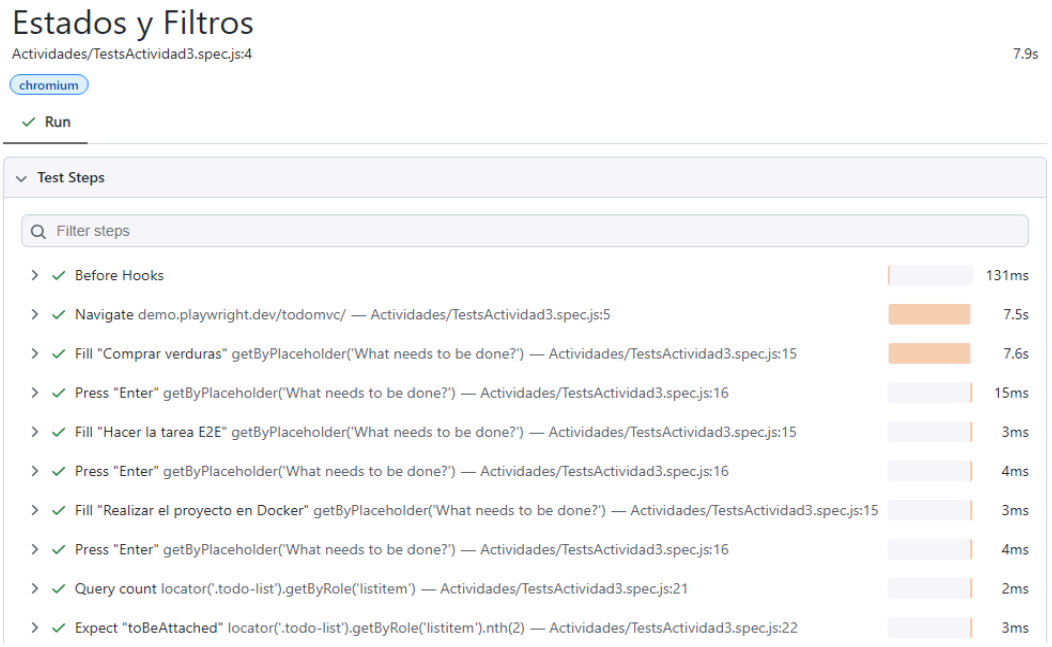
En esta segunda actividad se probará la creación de tareas en TodoMVC: añadir una tarea y comprobar que aparece, crear tres tareas consecutivas y verificar que se muestran, validar que el número de tareas es el correcto, comprobar el contenido de cada una, y ejecutar un caso negativo intentando crear una tarea vacía o formada solo por espacios.

Ha prueba pasa con éxito el test como se muestran en la imagen a continuación:



Actividad 3: Estados y filtros

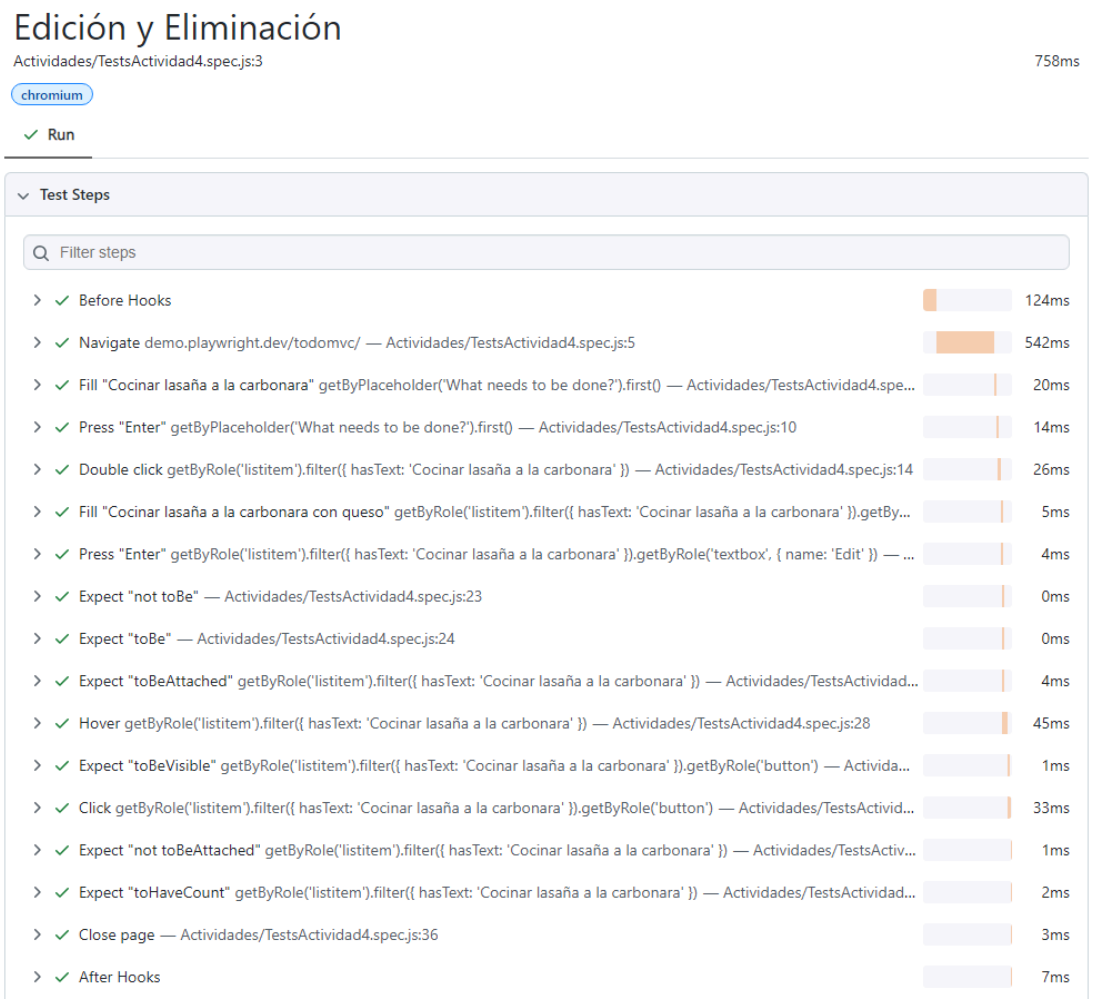
En este caso vamos a crear varias tareas donde se marcará una como completada, se verificará el cambio de estado y se comprobará el filtrado en Active, Completed y All, así como la actualización del contador de pendientes, usando las aserciones correspondientes. Vemos la imagen del resultado obtenido:



Actividad 4: Edición y eliminación

Continuamos con un test en el que crearemos una tarea que se editará con doble clic, se modificará y se guardará su contenido, se verificará el texto nuevo, se eliminará y se comprobará que desaparece.

El resultado del test lo podemos comprobar en la siguiente imagen:



Actividad 5: Verificación de errores y solución.

Finalmente, en este apartado, hemos modificado una de las pruebas realizadas en la actividad anterior.

Para provocar un fallo en la prueba, hemos modificado el código de la actividad 3. Los cambios realizados consisten en alterar las aserciones encargadas de verificar el estado de los checkboxes de las tareas:

- Se modifica la aserción para comprobar que las tareas pendientes están marcadas, cuando realmente sus checkboxes no han sido seleccionados.

- Del mismo modo, se modifica la aserción de las tareas completadas para verificar que no están marcadas. Esto provoca un error, ya que todas deberían tener el checkbox marcado (estado *true*)

```
// 4 - Seleccionamos el apartado de Activas y vemos las tareas pendientes.
await page.getByRole('link', {name: "Active"}).click();
const PENDIENTES = page.getByTestId('todo-item');
await expect(PENDIENTES).toHaveCount(tareas.length - 1);
for (let i = 0; i < await PENDIENTES.count(); i++) {
    const checkbox = PENDIENTES.nth(i).getByRole('checkbox');
    await expect(checkbox).toBeChecked();
}

// 5 - Seleccionamos el apartado de Completados y vemos las tareas completadas.
await page.getByRole('link', {name: "Completed"}).click();
const COMPLETADOS = page.getByTestId('todo-item');
await expect(COMPLETADOS).toHaveCount(1);
for (let i = 0; i < await COMPLETADOS.count(); i++) {
    const checkbox = COMPLETADOS.nth(i).getByRole('checkbox');
    await expect(checkbox).not.toBeChecked();
}
```

Errors

```
Error: expect(locator).toBeChecked() failed

Locator:  getByTestId('todo-item').first().getByRole('checkbox')
Expected: checked
Received: unchecked
Timeout:  5000ms

Call log:
  - Expect "toBeChecked" getByTestId('todo-item').first().getByRole('checkbox') with timeout 5000ms
  - waiting for getByTestId('todo-item').first().getByRole('checkbox')
    14 x locator resolved to <input class="toggle" type="checkbox" aria-label="Toggle Todo"/>
       - unexpected value "unchecked"


  31 |   await expect(PENDIENTES).toHaveCount(tareas.length - 1);
  32 |   for (let i = 0; i < await PENDIENTES.count(); i++) {
> 33 |       await expect(PENDIENTES.nth(i).getByRole('checkbox')).toBeChecked();
     |                                                             ^
  34 |   }
  35 |
  36 |   // 5 - Seleccionamos el apartado de Completados y vemos las tareas completadas.
    at C:\Users\Usuario\Documents\MASTER\ii\STWC\PR2\tests\TestsActividad5.spec.js:33:61
```

Copy prompt

```
Error: expect(locator).not.toBeChecked() failed

Locator:  getByTestId('todo-item').first().getByRole('checkbox')
Expected: not checked
Received: checked
Timeout:  5000ms

Call log:
  - Expect "not toBeChecked" getByTestId('todo-item').first().getByRole('checkbox') with timeout 5000ms
  - waiting for getByTestId('todo-item').first().getByRole('checkbox')
    14 x locator resolved to <input checked class="toggle" type="checkbox" aria-label="Toggle Todo"/>
       - unexpected value "checked"


  41 |   for (let i = 0; i < await COMPLETADOS.count(); i++) {
  42 |       const checkbox = COMPLETADOS.nth(i).getByRole('checkbox');
> 43 |       await expect(checkbox).not.toBeChecked();
     |                                   ^
  44 |   }
  45 |
  46 |   // 6 - Seleccionamos el apartado de Todas y vemos todas las tareas.
    at C:\Users\Usuario\Documents\MASTER\ii\STWC\PR2\tests\TestsActividad5.spec.js:43:34
```

Copy prompt

Observamos que los estados recibidos y esperados tienen un valor de verdad diferente tanto en la verificación de tareas pendientes como completadas.

```
// 4 - Seleccionamos el apartado de Activas y vemos las tareas pendientes.
await page.getByRole('link', {name: "Active"}).click();
const PENDIENTES = page.getByTestId('todo-item');
await expect(PENDIENTES).toHaveCount(tareas.length - 1);
for (let i = 0; i < await PENDIENTES.count(); i++) {
    const checkbox = PENDIENTES.nth(i).getByRole('checkbox');
    await expect(checkbox).not.toBeChecked();
}

// 5 - Seleccionamos el apartado de Completados y vemos las tareas completadas.
await page.getByRole('link', {name: "Completed"}).click();
const COMPLETADOS = page.getByTestId('todo-item');
await expect(COMPLETADOS).toHaveCount(1);
for (let i = 0; i < await COMPLETADOS.count(); i++) {
    const checkbox = COMPLETADOS.nth(i).getByRole('checkbox');
    await expect(checkbox).toBeChecked();
}
```

La solución a este problema consiste en corregir las aserciones, ajustando la condición que verifica el estado de los checkboxes:

- Para las tareas completadas, indicaremos que están marcadas (*toBeChecked()*), ya que previamente se ha hecho clic sobre ellas.
- Por el contrario, las tareas pendientes no tienen activada la propiedad checked del checkbox. Por lo tanto, esperamos que no estén marcadas (*.not.toBeChecked*).

Referencias

How can I check if a string is a valid number? (2008, 6 de octubre). Stack Overflow.

https://stackoverflow.com/questions/175739/how-can-i-check-if-a-string-is-a-valid-number

chrismead. (2022, 8 de marzo). Playwright - select child element innerText. Stack Overflow.

https://stackoverflow.com/questions/71402265/playwright-select-child-element-innertext

Array.prototype.filter() - JavaScript | MDN. (2025a, junio 24). https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

Array.prototype.filter() - JavaScript | MDN. (2025b, junio 24). https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

CodeKru. (2023, 18 marzo). *How to double click on an element using Playwright?* Codekru. https://www.codekru.com/playwright/how-to-double-click-on-an-element-using-playwright

Iebs, E. (2025, 10 noviembre). *Qué es XPath y como usarlo para localizar elementos en XML y HTML*. Thinking For Innovation. https://www.iebschool.com/hub/xpath-que-es-como-usarlo/

Locators | Playwright. (s. f.). https://playwright.dev/docs/locators#locate-by-label

Pathak, A. (2025, 10 diciembre). *getByRole in Playwright: Syntax and Examples | BrowserStack*. BrowserStack. https://www.browserstack.com/guide/playwright-getbyrole